



ವಿಶ್ವೇಶ್ವರಯ್ಯ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ, ಬೆಳಗಾವಿ
VISVESVARAYA TECHNOLOGICAL UNIVERSITY - BELAGAVI

A MINI PROJECT REPORT ON

“Network Intrusion detection using machine learning”

Submitted to Visvesvaraya Technological University in partial fulfillment of the requirement for the award of degree of

Bachelor of Engineering

in

Artificial Intelligence and Machine Learning.

Submitted by:

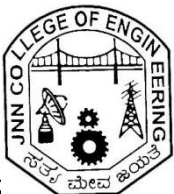
Ms. Adithi H S	4JN22AI001
Ms. Varshini S B	4JN22AI060
Ms. Pratheeksha K N	4JN23AI402

Under the Guidance of:

Mrs. Shaziya Banu S (Ph.D.)

Associate Professor,

Dept. of AIML



Department of Artificial Intelligence & Machine Learning

Jawaharlal Nehru New College of Engineering

Shivamogga - 577 204

National Education Society ®



Jawaharlal Nehru New College of Engineering
Department of Artificial Intelligence and Machine Learning
CERTIFICATE

This is to certify that the Mini-project Report entitled

**“NETWORK INTRUSION DETECTION USING MACHINE
LEARNING”**

Submitted by:

Ms. Adithi H S	4JN22AI001
Ms. Varshini S B	4JN22AI060
Ms. Prateeksha K N	4JN23AI402

Students of 5th semester B.E under the supervision and guidance towards the partial fulfillment of the requirement for award of degree of the Bachelor of Engineering in Artificial Intelligence and Machine Learning of Visvesvaraya Technological University, Belgaum during the year 2023–2024.

Dr. Chetan K R Ph.D

Professor and Head,
Dept. of AIML.

Mrs. Shaziya Banu S (Ph.D.)

Associate Professor,
Dept. of AIML.

Dr. Y Vijaya Kumar Ph.D.

Principal, JNNCE.

Acknowledgement

On presenting the Mini–Project report on “**Network Intrusion Detection using Machine Learning**”, we feel great to express our humble feelings of thanks to all those who have helped me directly or indirectly in the successful completion of the project work.

I would like to thank **Mrs. Shaziya Banu S**, Assistant Professor, Dept. of AIML, JNNCE, **Dr. Chetan K R**, Professor and Head, Dept. of AI&ML, JNNCE, Shivamogga and **Dr. Y Vijaya Kumar**, Principal, JNNCE, Shivamogga for all their support and encouragement.

I am grateful to Department of Artificial Intelligence and Machine Learning and our institution Jawaharlal Nehru New College of Engineering and for imparting me the knowledge with which I can do my best. Finally, I also would like to thank the whole teaching and non-teaching staff of the Artificial Intelligence and Machine Learning Dept.

Thanking you all,

Ms. Adithi H S	4JN22AI001
Ms. Varshini S B	4JN22AI060
Ms. Prateeksha K N	4JN23AI402

Abstract

The study proposes a model for building the network intrusion detection system using a machine learning algorithm called Gaussian NB. This system detects primarily an anomaly based intrusion. In this model, the categorical features from the dataset Change Control Identifiers (CCIDS) 2017 are encoded using label encoder. Using Correlation and selected columns method some best features is selected after reducing the column of the dataset. This data is then divided into training and testing data. Training data is then used to form a Gaussian NB Model where it signifies the possible outcome. Classification models are developed making use of the training data to classify the test data as malicious or benign. Measuring the accuracy of the classifier on future data rather than the past data is of a paramount aspect. The observed accuracy of the classifier on test data is 99%. The precision of the proposed system indicates that the True-Positive-Rate (TPR) is 99.9% and the False-Positive-Rate (FPR) is 0.1%. Moreover, unlike other systems, it does not use any data-mining tool like Weka. This work provides as basis for any new algorithm using dataset CCIDS 2017. Improvements: The work can be extended to exploit the big data available for attacks and intrusions using big data analytics.

Table of Contents

Acknowledgement	i
Abstract	ii
Table of Contents	iii
List of Figures	iv
List of Tables	v
CHAPTER 1 INTRODUCTION	1-4
1.1 Overview	1
1.2 Application	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Methodology	3
CHAPTER 2 DESIGN AND IMPLEMENTATION	5-11
2.1 Design	5
2.2 Implementation	9
2.3 APIs used	10
2.4 Summary	11
CHAPTER 3 RESULTS	12-20
3.1 Building of the dataset	12
3.2 Network dataset	12
3.3 Data pre-processing	13
3.4 Model training	14
3.5 Model testing	16
3.6 Development of frontend Interface for result display	17
3.7 Result	18
3.8 Summary	20
CHAPTER 4 CONCLUSION AND FUTURE SCOPE	21
REFERENCES	22

List of Figures

2.1	System Architecture	7
2.2	Implementation Flowchart	9
3.1	View of dataset	12
3.2	Various network dataset folder	13
3.3	Data pre-processing	14
3.4	Training model in Jupyter Model	15
3.5	Converted Jupyter Notebook to Python Script	15
3.6	Testing result	16
3.7	Flask Frontend VS code implementation	17
3.8	Result of network traffic detection	19

List of Tables

2.1	List of APIs	10
-----	--------------	----

Chapter-1

INTRODUCTION

1.1 Overview

The ever-increasing interconnectedness of today's digital landscape makes network security paramount. Sensitive information travels across these networks, making them attractive to hackers who exploit weaknesses. A critical component of modern cyber security strategies is the ability to identify and respond to anomalies in network traffic. This is where Intrusion Detection Systems (IDS) come in. IDS play a vital role in network security by detecting potential intruders. They operate across a broad spectrum, analyzing network or host system activity for malicious patterns.

1.2 Applications

- **Real-Time Threat Detection:** Identifies malicious activity, such as malware, unauthorized access, and suspicious traffic patterns in real-time.
- **Protection Against Cyberattacks:** Defends networks from Distributed Denial of Service (DDoS) attacks, phishing attempts, and brute force attacks.
- **Anomaly Detection:** Monitors for unusual behavior in network traffic to detect zero-day attacks and insider threats.
- **Data Protection:** Safeguards sensitive data by identifying and blocking attempts to access or exfiltrate information.
- **Compliance Assurance:** Helps meet regulatory requirements (e.g., GDPR, HIPAA) by monitoring and logging network activity.
- **Behavioral Analysis:** Tracks user and entity behaviors to establish baselines and identify deviations that indicate potential threats.
- **Protection in IoT and Smart Networks:** Detects vulnerabilities and attacks in IoT devices and smart systems, ensuring secure operation.

- **Traffic Monitoring and Analysis:** Provides insights into network usage, identifying bottlenecks or inefficiencies alongside security threats.

1.3 Problem Statement

The aim of the project is to develop an efficient and robust network intrusion detection system using machine learning techniques to identify and mitigate potential security threats in real-time. As cyberattacks become increasingly sophisticated, traditional rule-based detection methods struggle to adapt to the dynamic and evolving nature of malicious activities. This project seeks to leverage machine learning models to analyze network traffic, distinguish between normal and malicious activities, and proactively alert administrators about potential intrusions. By enhancing detection accuracy and reducing false positives, the system aims to strengthen network security and minimize the risks associated with cyber threats.

1.4 Objectives

1. Functional requirements:

- **Monitor and Analyze Network Traffic:** The system should continuously monitor network traffic to identify potential threats or malicious activities.
- **Leverage AI for Threat Detection:** Implement machine learning models to predict and classify intrusion attempts based on patterns in network data.
- **Provide Actionable Insights:** Generate detailed logs and reports on intrusion attempts for forensic analysis and security planning.

2. Non-functional requirements:

- **Scalability:** Ability to scale horizontally and vertically to accommodate growing network sizes and traffic volumes.
- **Accuracy:** High detection rate with minimal false positives and false negatives.
- **Availability:** Ensure high availability (99.9% uptime or more) to maintain continuous monitoring. Support fault tolerance and failover mechanisms.
- **Security:** Secure communication channels for alerts and logs (e.g., use of encryptions). Protect against tampering or attacks on the NIDS itself.

- **Performance:** Must process network traffic in real-time with minimal latency

1.5 Methodology

1. **Data Collection:** The CICIDS-2017 dataset, sourced from Kaggle, was used to simulate real-world network traffic for training and evaluating the Network Intrusion Detection System (NIDS). This dataset, created by the Canadian Institute for Cybersecurity, includes a diverse range of attack types (e.g., DoS, DDoS, Brute Force, and SQL Injection) alongside benign traffic. Data preparation involved cleaning, normalizing, and encoding the dataset's features, while balancing class distributions using techniques like SMOTE to address any imbalance. The dataset was divided into training (70%), testing (30%), and validation subsets, ensuring realistic, time-ordered traffic representation. These steps allowed the NIDS to be trained and tested on a robust, well-prepared dataset for effective intrusion detection.
2. **Preprocessing:** The data preprocessing of the CICIDS-2017 dataset involved cleaning and transforming it into a suitable format for machine learning. Numerical features were standardized using Standard Scalar to ensure a mean of 0 and a standard deviation of 1, which improved model performance by normalizing feature scales. Categorical data, such as protocol types and attack labels, were converted into integer values using label encoding to make them machine-readable. Additionally, the dimensionality of the dataset was reduced by selecting specific columns based on their relevance to network traffic analysis, such as flow duration, packet size, and protocol.
3. **Model Selection:** focuses on choosing appropriate machine learning models for the task. Different machine learning models like SVM, Gaussian NB techniques are explored for selecting a better model.
4. **Model Training:** After preprocessing the data, the training subset was fed into SVM and Gaussian NB () models to learn patterns differentiating benign and malicious traffic. SVM was employed due to its effectiveness in handling high-dimensional data. Gaussian Naïve Bayes, a probabilistic classifier, was chosen for its simplicity and efficiency in modelling feature likelihoods assuming a Gaussian distribution. Both models were trained and evaluated on the dataset using metrics like accuracy, precision, recall, and F1-score to assess their performance.

5. **Evaluation:** Evaluation of the trained models is conducted using metrics such as accuracy, precision, recall, and F1-score. A confusion matrix is also used to analyze the model's performance on different gestures, identifying areas where the model excels or needs improvement. This comprehensive methodology ensures that the network intrusion detection system is accurate and reliable, significantly enhancing the safety in cyber security.

Chapter-2

DESIGN AND IMPLEMENTATION

2.1 Design

Designing a Network Intrusion Detection System (NIDS) involves creating a framework capable of monitoring, detecting, and responding to unauthorized or malicious activities within a network. The system leverages advanced data analysis, anomaly detection, and machine learning techniques to identify potential threats effectively. It integrates seamlessly with network infrastructure, ensuring real-time detection and response to mitigate security risks. The system typically includes components for data collection, preprocessing, analysis, and visualization, focusing on user-friendliness and operational efficiency to safeguard network resources.

2.1.1 Design Issues

- **Scalability:** Ensuring the system can handle large-scale networks and high traffic volumes.
- **Accuracy:** Minimizing false positives and false negatives to ensure reliable detection of intrusions.
- **Real-time Performance:** Achieving quick processing to detect and respond to threats promptly.
- **Diverse Threat Detection:** Supporting various attack vectors, including known and zero-day threats.
- **Integration:** Ensuring compatibility with existing network infrastructure and security tools.
- **Data Security:** Safeguarding sensitive network data collected by the system.
- **Resource Optimization:** Designing the system for efficient resource usage, particularly on edge devices.
- **User Interface:** Creating an intuitive interface for security analysts to monitor and respond to threats effectively.

2.1.2 Assumptions

- The system operates in a well-maintained and standardized network environment.

- Network traffic is sufficiently monitored and logged for analysis.
- The dataset used for training the detection models includes diverse and updated attack patterns.
- Administrators and security analysts are trained to interpret alerts and take appropriate actions.
- The system supports and is trained to detect the most common types of intrusions (e.g., DDoS attacks, phishing, malware).
- The network has minimal noise, and traffic patterns are consistent for effective anomaly detection.
- Organizations are willing to allocate resources for hardware, software, and training for the system.
- The system is regularly updated with new threat signatures and machine learning models.

2.1.3 System Architecture

The architecture of a Network Intrusion Detection System (NIDS) consists of several interconnected modules designed to ensure seamless detection and response to network threats. It begins with a Data Collection Module, which captures raw network traffic data using packet sniffers or log collection tools. The Preprocessing Module refines this data by filtering out noise and normalizing it for consistency, preparing it for analysis. Next, the Feature Extraction Module identifies critical attributes such as packet size, protocol types, and connection patterns, which are essential for distinguishing normal traffic from anomalies. This data is then fed into the Model Training Module, where machine learning algorithms or rule-based techniques are used to develop intrusion detection models. The Model Evaluation Module tests these models using a separate dataset to ensure their accuracy and reliability. In live scenarios, the Intrusion Detection Module applies the trained model to monitor real-time network traffic, identifying and classifying potential threats. Detected intrusions are handled by the Response Module, which triggers alerts or initiates actions such as blocking malicious traffic. Finally, the Visualization and Reporting Module provides security analysts with graphical insights and detailed reports, enabling effective monitoring and decision-making.

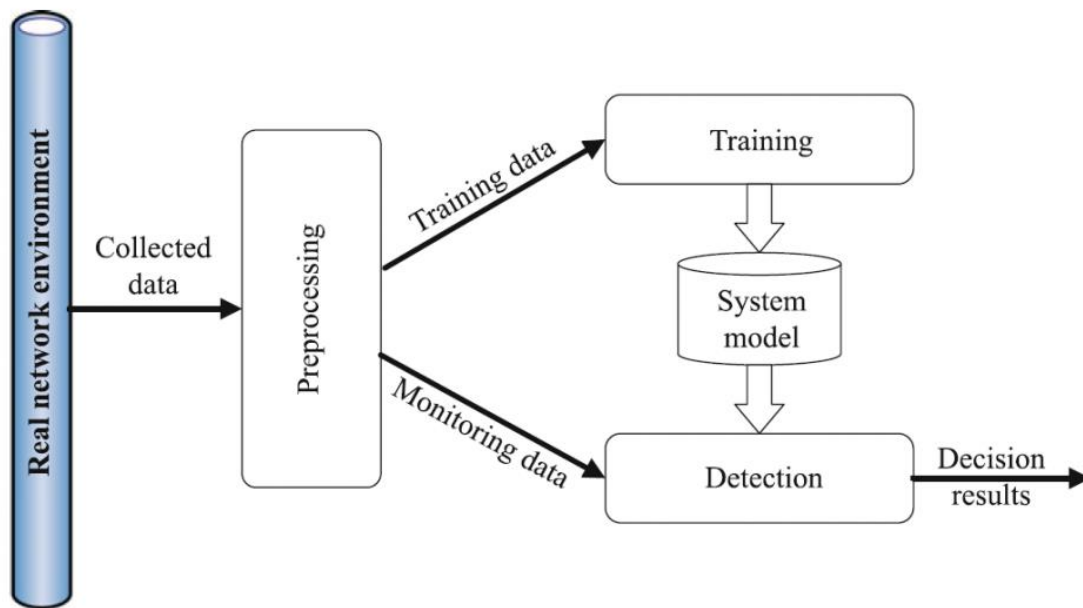


Fig 2.1 System Architecture

The above Fig 2.1 gives the System architecture of the system, architecture gives the information about how the system works.

The dataset serves as the foundation for building and training the Network Intrusion Detection System (NIDS). It includes labelled network traffic data, with examples of normal and malicious activities. Public datasets such as CICIDS-2017 are often used for training and testing purposes. This stage involves cleaning and transforming the raw network traffic data into a suitable format for analysis. It includes tasks like removing noise, handling missing values, normalizing features, and converting categorical data into numerical values to ensure consistency and improve model accuracy.

Feature selection techniques are applied to identify the most relevant attributes (features) from the dataset that significantly contribute to detecting network intrusions. This step reduces dimensionality, improves computation efficiency, and enhances model performance by focusing on critical features such as packet size, protocol type, and traffic behaviour. Class imbalance is common in intrusion detection datasets, where normal traffic dominates over malicious activities. Class balancing techniques such as oversampling (e.g., SMOTE) or under sampling ensure that the training dataset has a balanced distribution of classes, which helps prevent model bias and improves the detection of minority class attacks.

The training dataset is a subset of the overall data that is used to train the intrusion detection model. This data consists of labelled examples, where each record is categorized as normal or a specific type of attack, enabling the model to learn and classify new inputs. The

testing dataset is a separate subset of the data that is used to evaluate the performance of the trained model. It ensures that the model generalizes well to unseen data and measures its ability to correctly classify network traffic.

Classification algorithms, such as Decision Trees, Support Vector Machines (SVM), Random Forests, or Deep Learning techniques (e.g., Neural Networks), are applied to build the intrusion detection model. These algorithms use the training dataset to learn patterns and differentiate between normal and malicious traffic. The "Gaussian naïve bayes Model" represents the trained intrusion detection system after applying classification algorithms. It encapsulates the learned patterns and decision rules required to classify new, unseen network traffic into normal or attack categories. This step involves evaluating the trained model's performance using various metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. These metrics help determine the model's effectiveness in detecting intrusions and minimizing false positives and false negatives.

At this stage, the system checks whether the model achieves acceptable performance based on predefined thresholds for accuracy and other evaluation metrics.

- If No: The process iterates back to the Feature Selection Techniques or Class Balancing Techniques to refine the model.
- If Yes: The model proceeds to real-time prediction.

Predicting Network Attack Category Once the model achieves acceptable performance, it is deployed to predict and classify network traffic into various attack categories (e.g., DoS, Probe, R2L, U2R) or normal behaviour in real-time. This stage enables timely detection of malicious activities and helps secure the network.

2.2 Implementation

This flowchart provides a detailed visualization of the workflow for implementing the Gaussian Naive Bayes (GNB) algorithm in network intrusion detection. The process begins with the collection of N-dimensional network traffic data samples, which include both normal and malicious traffic. Once the data is pre-processed and reduced, it is split into two subsets: training data and testing data. The training data is fed into the Gaussian Naive Bayes classifier, a probabilistic algorithm well-suited for classifying network traffic based on Gaussian (normal) distribution assumptions.

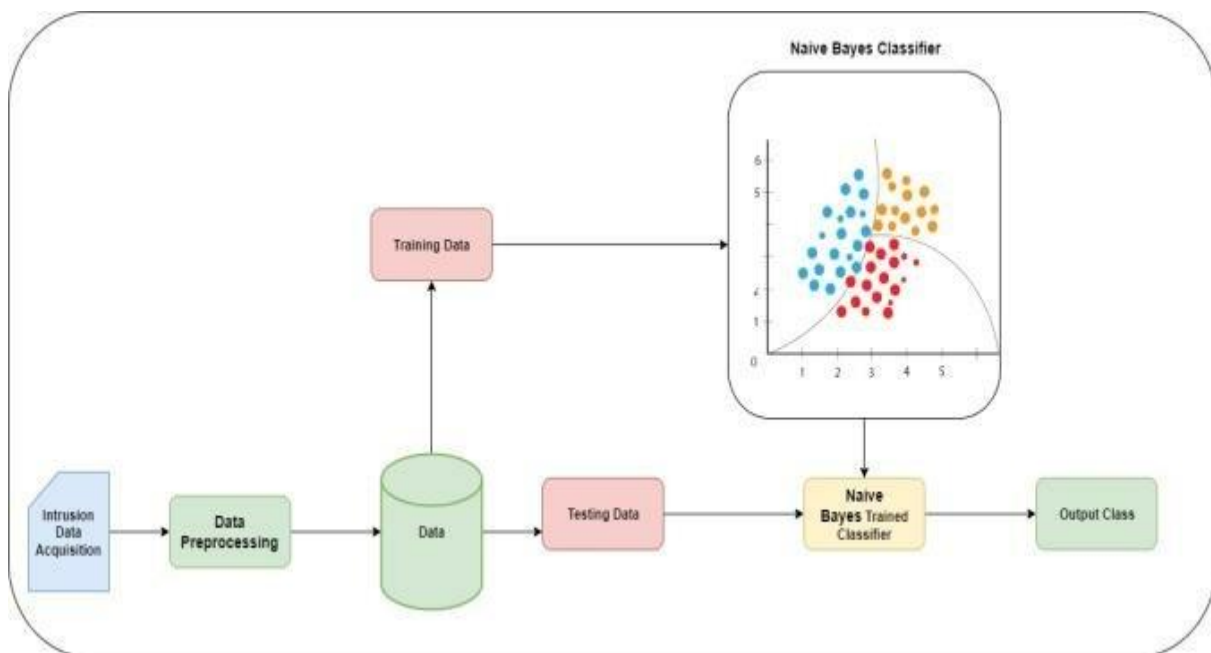


Figure 2.2 Implementation Flowchart

The GNB algorithm learns the relationships between features and class labels (e.g., normal traffic, DoS attacks, etc.), resulting in a trained intrusion detection model. This trained model is then validated using the testing data to assess its performance. The evaluation is performed using standard metrics such as accuracy, precision, recall, and F1-score to measure how well the model distinguishes between normal and malicious traffic. If the comparative analysis shows that the model's performance is inadequate (e.g., low accuracy or high false positives), the process loops back to the training phase. Additional refinement steps may include further feature selection or adjusting the input data to improve the model's effectiveness. Finally, once the desired accuracy and performance levels are achieved, the model undergoes a final performance evaluation to ensure it meets the specified requirements. The validated model can then be deployed to predict and classify real-time network traffic into predefined categories. This ensures the system delivers reliable intrusion detection, contributing to enhanced network security.

2.3 APIs used

API/Library	Description
Flask @app.route	The @app.route decorator in Flask is used to define a route that maps a specific URL to a corresponding function, enabling the function to handle requests to

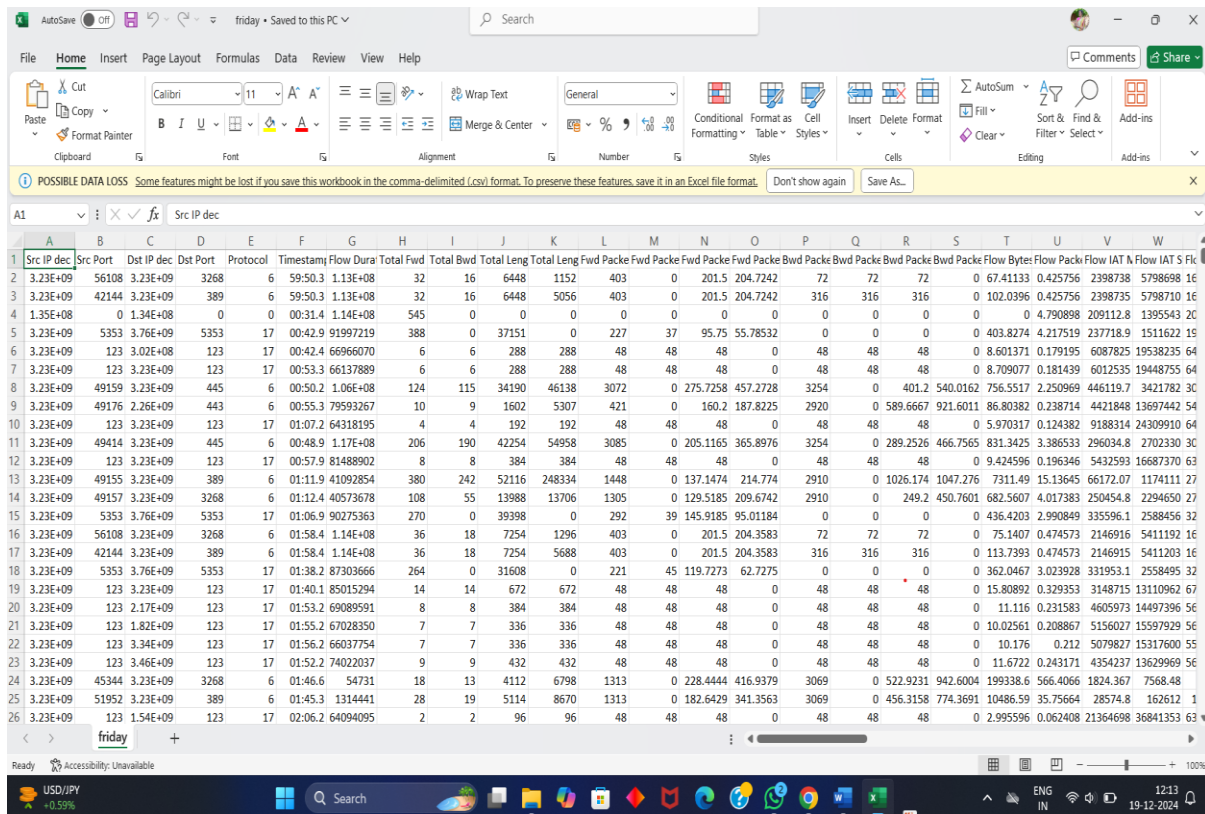
	that URL.
Jupyter Notebook	An interactive environment for coding, debugging, and visualizing NIDS workflows, data analysis, and results.
NumPy	A library for numerical operations in Python, used for array manipulation and data processing.
Pandas	A data manipulation library used for organizing and processing network traffic datasets.
sklearn (scikit-learn)	A machine learning library used to implement and train classification algorithms such as Gaussian Naive Bayes.
model.fit()	This function in the Naive Bayes algorithm is used to train the model by calculating the probabilities based on the features and labels provided in the training dataset.
model.predict()	This function in a Naive Bayes algorithm is used to predict the class labels for given input data based on the probabilities calculated during the model's training phase.
Model_selection()	This function in Naive Bayes algorithms is typically used to split the dataset into training and testing subsets, helping to evaluate the model's performance on unseen data.

Table 2.1 List of APIs

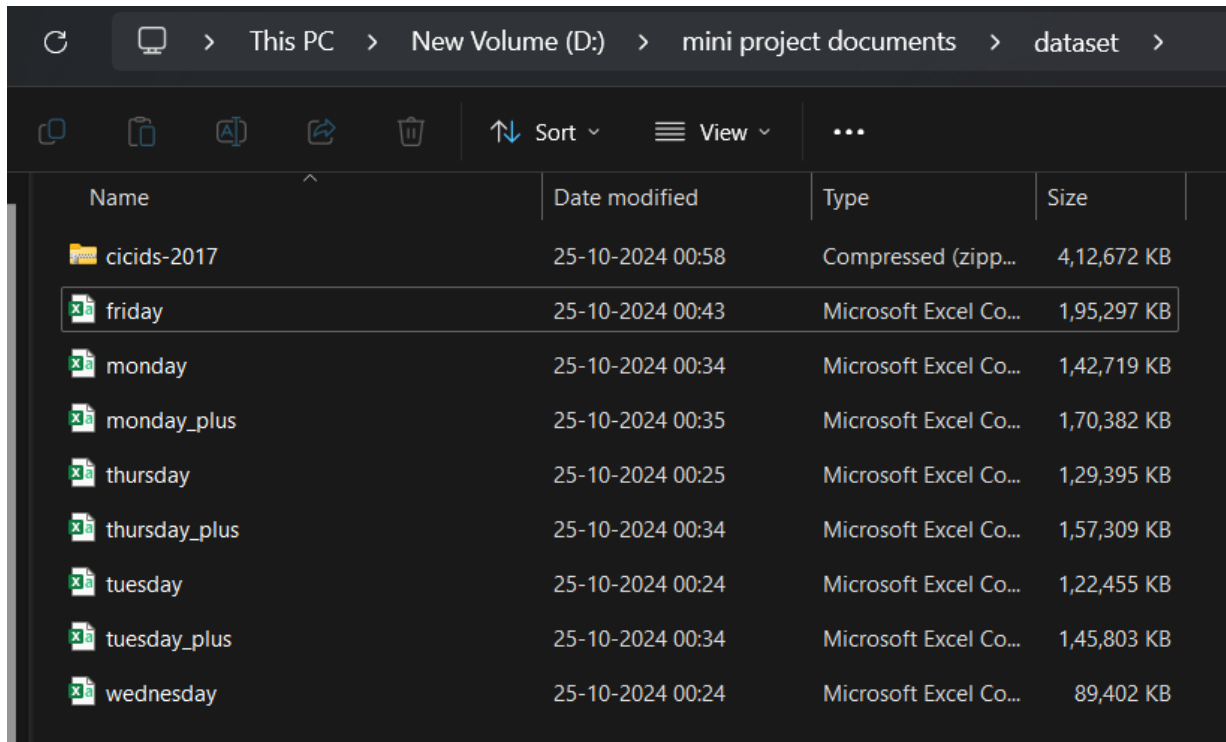
2.4 Summary

This chapter describes the design principles, assumptions and the implementation of the system. The building of the system requires some of the APIs and these are mentioned in the table and the figures representing the implementation procedures. In the next chapter the results of the projects are shown.

RESULTS



(NIDS). It includes both normal and malicious traffic, with categories such as DoS attacks, phishing attempts, SQL injection, and port scanning. Each data sample comprises key network features like source and destination IPs, protocol types, packet sizes, and timestamps, stored in .pcap or .csv formats. The data is organized into folders for training, testing, and validation, ensuring a clear separation for model development and evaluation. This comprehensive dataset provides a realistic and diverse representation of network activity, enabling robust machine learning model training for intrusion detection.



Name	Date modified	Type	Size
cicids-2017	25-10-2024 00:58	Compressed (zip)...	4,12,672 KB
friday	25-10-2024 00:43	Microsoft Excel Co...	1,95,297 KB
monday	25-10-2024 00:34	Microsoft Excel Co...	1,42,719 KB
monday_plus	25-10-2024 00:35	Microsoft Excel Co...	1,70,382 KB
thursday	25-10-2024 00:25	Microsoft Excel Co...	1,29,395 KB
thursday_plus	25-10-2024 00:34	Microsoft Excel Co...	1,57,309 KB
tuesday	25-10-2024 00:24	Microsoft Excel Co...	1,22,455 KB
tuesday_plus	25-10-2024 00:34	Microsoft Excel Co...	1,45,803 KB
wednesday	25-10-2024 00:24	Microsoft Excel Co...	89,402 KB

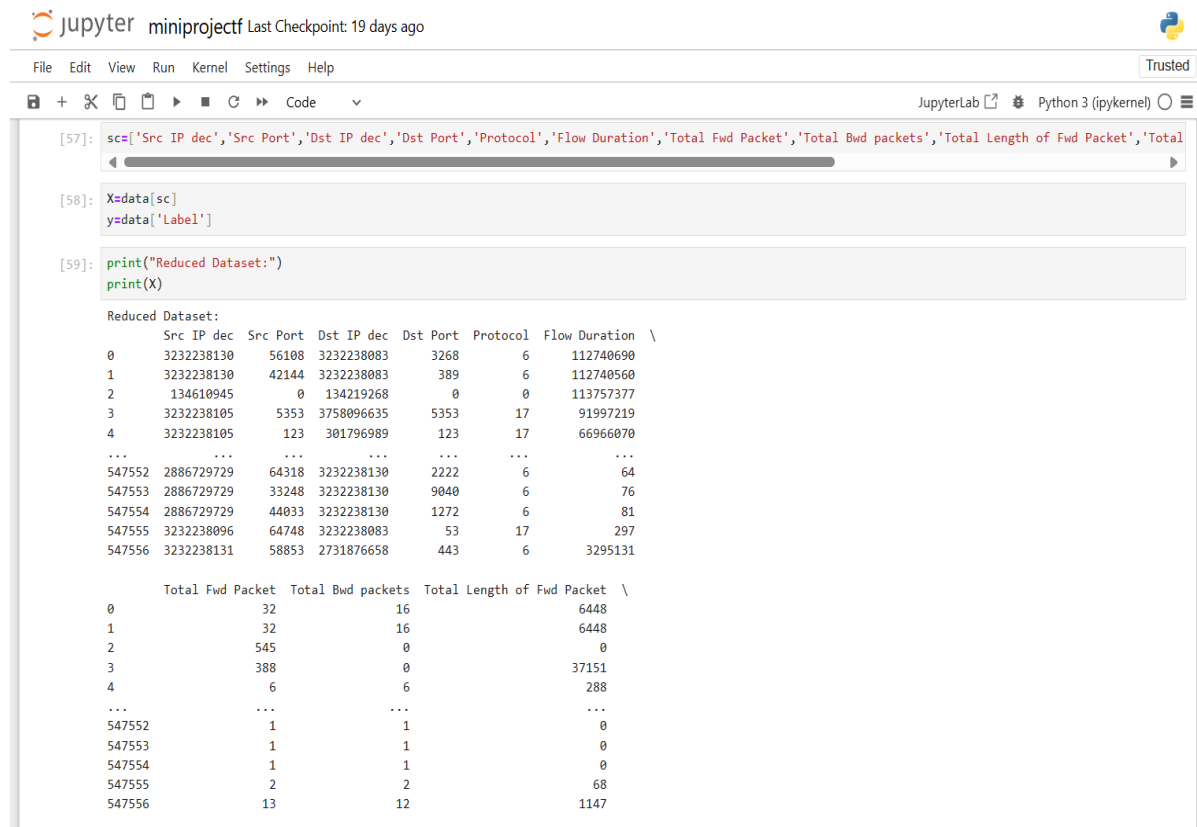
Fig 3.2 Various network dataset folder

3.3 Data pre-processing

Data preprocessing is a critical step in building a Network Intrusion Detection System (NIDS) as it ensures that the dataset is optimized for efficient and accurate model training. One of the key preprocessing techniques involves reducing the number of columns in the dataset by selecting only the most relevant features. This process, known as feature selection or dimensionality reduction, is performed to eliminate irrelevant, redundant, or highly correlated columns that do not significantly contribute to the model's performance.

By focusing on the most significant features, the reduced dataset enhances the performance of the intrusion detection model. During preprocessing, iterative experiments are

conducted to ensure that the selected columns preserve the critical patterns and relationships necessary for distinguishing between normal and malicious traffic. This streamlined dataset allows the machine learning algorithm to generalize better on unseen data, improving its ability to detect intrusions accurately.



```
[57]: sc=['Src IP dec','Src Port','Dst IP dec','Dst Port','Protocol','Flow Duration','Total Fwd Packet','Total Bwd packets','Total Length of Fwd Packet','Total

[58]: X=data[sc]
      y=data['Label']

[59]: print("Reduced Dataset:")
      print(X)
```

Reduced Dataset:

	Src IP dec	Src Port	Dst IP dec	Dst Port	Protocol	Flow Duration \
0	3232238130	56108	3232238083	3268	6	112740690
1	3232238130	42144	3232238083	389	6	112740560
2	134610945	0	134219268	0	0	113757377
3	3232238105	5353	3758096635	5353	17	91997219
4	3232238105	123	301796989	123	17	66966070
...	...	...	...	...	...	...
547552	2886729729	64318	3232238130	2222	6	64
547553	2886729729	33248	3232238130	9040	6	76
547554	2886729729	44033	3232238130	1272	6	81
547555	3232238096	64748	3232238083	53	17	297
547556	3232238131	58853	2731876658	443	6	3295131

	Total Fwd Packet	Total Bwd packets	Total Length of Fwd Packet \
0	32	16	6448
1	32	16	6448
2	545	0	0
3	388	0	37151
4	6	6	288
...	...	...	...
547552	1	1	0
547553	1	1	0
547554	1	1	0
547555	2	2	68
547556	13	12	1147

Fig 3.3 Data pre-processing

3.4 Model training

The model training process employs the Gaussian Naïve Bayes algorithm, implemented using Python in a Jupyter Notebook environment. The training dataset, consisting of extracted features from one week of network traffic data, is pre-processed by normalizing numerical values and encoding categorical variables. The Gaussian Naïve Bayes model is chosen for its simplicity and effectiveness in handling continuous data, making it well-suited for network intrusion detection tasks. The training process involves splitting the dataset into training and validation subsets, fitting the model to the training data, and evaluating its performance using metrics such as accuracy, precision, recall, and F1-score. The implementation leverage libraries like Scikit-learn to streamline data processing and model training, with the results displayed in real-time for analysis.

jupyter miniprojectf Last Checkpoint: 19 days ago

File Edit View Run Kernel Settings Help

+ ✂ 📄 📌 ▶ ■ ↺ ⏪ Code ▾

```
[11]: from sklearn.model_selection import train_test_split
```

```
[12]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
[13]: print("Training Features:", X_train)
```

```

Training Features:
11888 3232238105 50067 3107018018 80 6 64074208 \
381846 2886729729 55191 3232238130 5225 6 44
367981 2886729729 56932 3232238130 2601 6 75
100970 2886729729 62927 3232238130 80 6 10359717
100467 2886729729 62505 3232238130 80 6 1987961
...
110268 2886729729 50783 3232238130 80 6 8028093
259178 2886729729 56426 3232238130 1098 6 40
365838 2886729729 41856 3232238130 1117 6 49
131932 2886729729 63135 3232238130 80 6 12026832
121958 2886729729 55936 3232238130 80 6 3241950

Total Fwd Packet Total Bwd packets Total Length of Fwd Packet \
11888 2 0 0
381846 1 1 0
367981 1 1 0
100970 8 6 20
100467 8 5 20
...
110268 10 7 20
259178 1 1 0
365838 1 1 0
131932 7 5 20
121958 8 6 20

Total Length of Bwd Packet ... Fwd Act Data Pkts Fwd Seg Size Min \
11888 0 ... 0 32
381846 0 ... 0 24
367981 0 ... 0 40
100970 11595 ... 1 20

```

Fig 3.4 Training model in Jupyter notebook

File Edit Selection View Go Run Terminal Help

EXPLORER

- botscheck
 - ipynb_checkpoints
 - templates
 - app.py
 - bots1.py
 - friday.csv
 - miniproject.ipynb
 - nb_model.pkl

bots1.py

```

1 import pandas as pd
2 from sklearn.svm import SVC
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.naive_bayes import GaussianNB
7 from sklearn.metrics import accuracy_score
8
9
10
11 file_path = r"friday.csv"
12 data = pd.read_csv(file_path)
13 X = data.drop(columns=["Label"])
14 y = data["Label"]
15 X = data.select_dtypes(include='int64')
16 y = y.astype('category').cat.codes
17 sc= ['Src IP dec', 'Src Port', 'Dst IP dec', 'Dst Port', 'Protocol', 'Flow Duration', 'Total Fwd Packet', 'Total Bwd packets', 'Total Length of Fw
18 X=data[sc]
19 y=data["Label"]
20
21 Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, test_size=0.3, random_state=42)
22
23
24 model = GaussianNB()
25 model.fit(Xtrain, ytrain)
26 ypred=model.predict(Xtest)
27 print(ypred)
28
29
30 acc=accuracy_score(ypred, ytest)
31 print(acc)
32
33 with open('nb_model.pkl', 'wb') as f:
34     pickle.dump(model, f)

```

Julia: Downloading cache files... (75%) Julia env: v1.10

Fig 3.5 Converted Jupyter Notebook to Python Script

When a Jupyter Notebook is converted into a Python script, the interactive cells in the notebook are transformed into a linear Python code file with comments denoting the original cell structure. The script includes all code, markdown, and outputs in a sequential format, making it executable in any Python IDE or terminal. This format is particularly useful for sharing or deploying models, as it ensures compatibility outside the notebook environment while retaining the logical flow of the code. The resulting script retains all functionality, such as data preprocessing, model training, and evaluation.

3.5 Model testing

The testing phase involves evaluating the performance of the trained Gaussian Naïve Bayes model using unseen data from the test dataset. This dataset contains labelled network traffic samples, including both normal and malicious activities. During testing, the model predicts the labels for the test data based on its training. Key performance metrics such as accuracy, precision, are calculated to assess the model's effectiveness in detecting network intrusions. The confusion matrix provides a detailed view of true positives, true negatives, false positives, and false negatives, highlighting the model's strengths and areas for improvement.

```
[73]: model.fit(Xtrain,ytrain)

[73]: GaussianNB ⓘ ⓘ
      GaussianNB()

[74]: ypred=model.predict(Xtest)
      ypred

[74]: array(['BENIGN', 'BENIGN', 'BENIGN', ..., 'BENIGN', 'BENIGN', 'DDoS'],
      dtype='<U18')

[75]: from sklearn.metrics import accuracy_score
      accuracy_score(ypred,ytest)

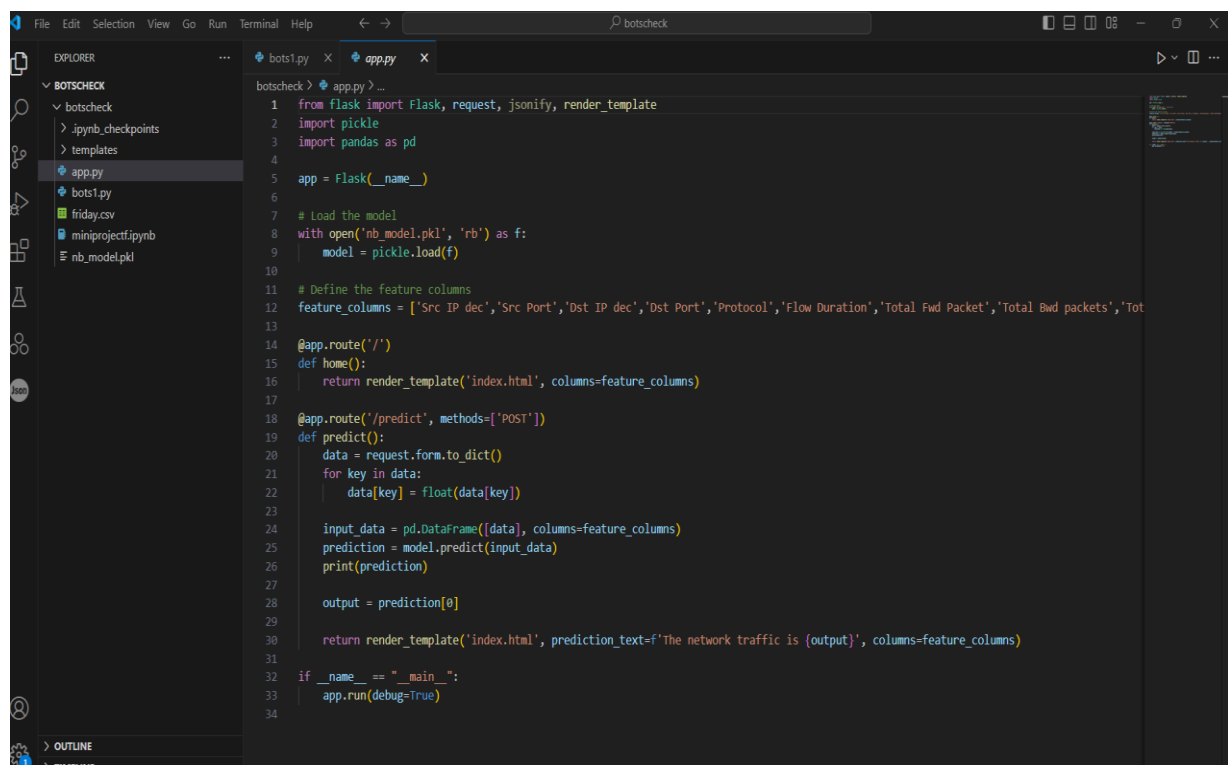
[75]: 0.9904546229332554
```

Fig 3.6 Testing Result

The testing phase of the Network Intrusion Detection System (NIDS) involved evaluating the Gaussian Naive Bayes model's performance on a separate test dataset that was not used during training. This phase ensures the model's ability to generalize to new, unseen network traffic data. The test dataset included both normal and malicious traffic, carefully labelled to facilitate accurate comparison with the model's predictions. The model achieved an impressive accuracy of 99%, indicating its robustness in distinguishing between normal behaviour and various types of network intrusions.

The high accuracy of the model demonstrates the effectiveness of preprocessing steps, feature selection, and the Gaussian Naive Bayes algorithm in capturing the patterns and nuances of network traffic. A confusion matrix revealed that the model made only a few misclassifications, reflecting its precision in detecting anomalies. Additionally, the testing phase included visual analyses such as ROC curves to validate the trade-off between true positive and false positive rates. The results confirm that the NIDS is well-suited for deployment in real-world scenarios, where accurate and timely detection of intrusions is critical. The successful testing with 99% accuracy establishes a solid foundation for further enhancements and scaling of the system to handle larger, more complex datasets.

3.6 Development of Front-End Interface for Results Display



```
1 from flask import Flask, request, jsonify, render_template
2 import pickle
3 import pandas as pd
4
5 app = Flask(__name__)
6
7 # Load the model
8 with open('nb_model.pkl', 'rb') as f:
9     model = pickle.load(f)
10
11 # Define the feature columns
12 feature_columns = ['Src IP dec', 'Src Port', 'Dst IP dec', 'Dst Port', 'Protocol', 'Flow Duration', 'Total Fwd Packet', 'Total Bwd packets', 'Tot
13
14 @app.route('/')
15 def home():
16     return render_template('index.html', columns=feature_columns)
17
18 @app.route('/predict', methods=['POST'])
19 def predict():
20     data = request.form.to_dict()
21     for key in data:
22         data[key] = float(data[key])
23
24     input_data = pd.DataFrame([data], columns=feature_columns)
25     prediction = model.predict(input_data)
26     print(prediction)
27
28     output = prediction[0]
29
30     return render_template('index.html', prediction_text=f'The network traffic is {output}', columns=feature_columns)
31
32 if __name__ == '__main__':
33     app.run(debug=True)
34
```

Fig 3.7 Flask Frontend VS Code Implementation

The front end of the Network Intrusion Detection System (NIDS) was developed using Flask, a lightweight web framework in Python. Flask was chosen for its simplicity and flexibility, allowing the creation of a responsive web interface to interact with the trained machine learning model. The front end enables users to upload network traffic data for real-time intrusion detection. Once the data is uploaded, Flask processes it and sends it to the backend, where the pre-trained Gaussian Naive Bayes model makes predictions on whether the traffic is normal or malicious. The results are then displayed on the web interface, providing immediate feedback to the user. The Flask application is built with HTML, CSS, and JavaScript to ensure an intuitive and user-friendly design, offering seamless interaction with the underlying model. This integration of Flask with the machine learning model provides a simple yet effective means for deploying the NIDS in a practical, real-world environment.

3.7 Results

The result web page of the Network Intrusion Detection System (NIDS) serves as an interactive interface for users to input specific network data and receive predictions regarding potential intrusions or threats. This page is designed with user accessibility and clarity in mind, featuring a structured layout that facilitates seamless data entry and intuitive understanding of results.

Key parameters include the source and destination IP addresses (in decimal format), source and destination port numbers, protocol type, flow duration, total number of forward and backward packets, cumulative length of these packets, and the maximum and minimum lengths of forward packets. These fields capture crucial aspects of network traffic, enabling the system to comprehensively analyze data for accurate predictions.

Once the input is provided, users can click the "Predict" button to initiate the detection process. The backend system, likely powered by a machine learning model or a rule-based algorithm, processes the data to classify the network activity. The prediction result is displayed prominently at the bottom of the page. In the example shown, the system predicts that "The network traffic is DDoS", indicating that the provided network data corresponds to a Distributed Denial of Service attack. The output is displayed in a simple, visually distinct format, ensuring clarity and quick interpretation.

This result interface is both user-friendly and functional. Its minimalist design ensures ease of use, while the sequential arrangement of input fields creates an intuitive workflow for

data entry. The visual separation of the prediction output further enhances usability by allowing users to focus on the results effortlessly.

Network Traffic Prediction

Src IP dec:

Src Port:

Dst IP dec:

Dst Port:

Protocol:

Flow Duration:

Total Fwd Packet:

Total Bwd packets:

Total Length of Fwd Packet:

Total Length of Bwd Packet:

Fwd Packet Length Max:

Fwd Packet Length Min:

Predict

The network traffic is DDoS

Figure 3.8 Result of network traffic detection

The NIDS result interface is invaluable in real-world scenarios. It enables network administrators to detect anomalies and threats in real-time, providing actionable insights to mitigate risks. By identifying malicious activities like DDoS, Botnet, Botnet-Attempted and Portscan attacks, the system helps secure networks and prevent potential disruptions. Moreover, this interface can also serve as a training or research tool for studying intrusion detection techniques and understanding network behaviours.

Future enhancements to the interface could include additional features such as confidence scores for predictions, detailed explanations of detected threats, and a log for tracking previous results. Tooltips or descriptions for each input field and access to documentation would further improve usability, especially for new users. Overall, this web page is a critical component of a NIDS, providing a seamless platform for monitoring network security and detecting intrusions efficiently.

3.8 Summary

This chapter describes the result of the project which was aimed at the beginning of the project and mentioned in the problem statement section. In the next chapter the conclusion and the Future scope of the project is written.

Chapter 4

CONCLUSIONS AND FUTURE SCOPE

Implementing a network intrusion detection system (NIDS) significantly enhances network security and ensures a safer and more reliable environment for organizations in the face of evolving cyber threats. By leveraging advanced technologies such as Scikit-learn or TensorFlow for training robust machine learning models, and Flask/Django for seamless API integration, organizations can effectively detect and mitigate potential intrusions in real-time. These technologies enable comprehensive traffic monitoring, anomaly detection, and automated threat classification, reducing the reliance on manual monitoring and improving response time. Beyond improving security, NIDS fosters organizational resilience by minimizing downtime, protecting sensitive information, and maintaining trust among customers and stakeholders setting a new standard for network security solutions.

4.1 Future Scope

Network Intrusion Detection Systems (NIDS) have a promising future as cybersecurity threats grow in complexity and volume. With advancements in AI and machine learning, NIDS are evolving to provide real-time, adaptive, and more accurate threat detection. The integration of NIDS with cloud security, IoT networks, and 5G infrastructure further broadens their applications, ensuring robust protection for modern and future digital ecosystems.

REFERENCES

- [1]. Jamadar RA, Himani Gupta, “Ankit Baghel & Rituraj. Study and analysis of hadoop based Network Intrusion Detection System”, International Journal of Engineering and Science Invention. Dec 2017; 6(12):1–4.
- [2]. Kumar S. Viinikainen A. & Hamalainen T. “Machine learning classification model for network based intrusion detection system”, 2016 11th International Conference for Internet Technology and Secured Transactions (ICITST), 5-7 Dec. 2016. IEEE, Barcelona, Spain; Dec 2016.
- [3]. Divyatmika & Manasa Sreekesh. “A Two-tier Network based Intrusion Detection System Architecture using Machine Learning Approach”. 2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT), 3-5 March 2016. IEEE, Chennai, India; Mar. 2016. p. 42–47.
- [4]. Jamal Esmaily, Reza Moradinezhad & Jamal Ghasemi. “Intrusion Detection System Based on Multi-Layer Perceptron Neural Networks and Decision Tree”. 2015 7th Conference on Information and Knowledge Technology (IKT), 05 October 2015.
- [5]. Ali MH, Bahaa Abbas Dawood Al Mohammed, Alyani Ismail & Mohamad Fadli Zolkipli. “A new intrusion detection system based on Fast learning network and swarm optimization” IEEE, 2018.
- [6]. Chabathula KJ, Jaidhar CD & Ajay Kumara MA. “Comparative study of principal component analysis-based intrusion detection approach using machine learning Algorithms”. 3rd International Conference on Signal Processing, Communication and Networking (ICSCN); 2015.
- [7]. Effendy DA, Kusrini & Sudarmawan. “Classification of intrusion Detection System (IDS) based on computer network”. 2017 2nd International conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE). IEEE, Yogyakarta: Indonesia; 2017. p. 90–94.
- [8]. “Mathematics Behind Classification and Regression”. Date accessed: 26.11.2012. [questions/44382/mathematics-behind-classification-and-regression-trees](https://www.khanacademy.com/a/mathematics-behind-classification-and-regression-trees/a/mathematics-behind-classification-and-regression-trees).